- [ ] Update GitHub repo for Deliverable #2 & add files
- [x] ~~Write README / Deliverable #2 commit (Raisa Reza)~~
- [x] ~~Report title (Jaden Kuriakose)~~
- [ ] Task delegation section
    - [x] ~~Jaden Kuriakose~~
    - [ ] Dev Sheth
    - [ ] Dev Singh
    - [x] ~~Abhinav Binu~~
    - [x] ~~Ahmed Fouad~~
    - [x] ~~Raisa Reza~~
- [x] ~~Include updated Deliverable #1 content~~
    - [x] ~~Improve Functional Requirements (Jaden Kuriakose)~~
    - [x] ~~Fix Use Case Diagrams (Raisa Reza)~~
    - [x] ~~Fix Sequence Diagrams (Abhinav Binu)~~
- [x] ~~Project scheduling / timeline (Raisa Reza)~~
- [x] ~~Cost & effort estimation method (Function Point or Application Composition) (Dev Singh)~~
- [x] ~~Hardware cost estimation (Abhinav Binu)~~
- [x] ~~Software cost estimation (Jaden Kuriakose)~~
- [x] ~~Personnel cost estimation (Abhinav Binu)~~
- [x] ~~Test plan for one software unit - pdf extraction (Jaden Kuriakose)~~
- [x] ~~Unit implementation code - pdf extraction (Jaden Kuriakose)~~
- [x] ~~Automated unit tests(JUnit or similar) (Ahmed Fouad)~~
    - [x] ~~pytests will be for the pdf extraction function underneath project schedule~~
- [x] ~~Test cases & test results documentation (Dev Singh)~~
- [x] ~~Comparison with similar systems / designs (Jaden Kuriakose)~~
- [x] ~~Prototype of application (Figma) (Raisa Reza)~~
- [x] ~~Conclusion / project evaluation (Raisa Reza)~~
- [x] ~~References (IEEE format) (Team Effort)~~
- [x] ~~Presentation slides~~
    - [x] ~~Title (Jaden Kuriakose)~~
    - [x] ~~Objectives (Jaden Kuriakose)~~
    - [x] ~~Cost Estimation (Ahmed Fouad)~~
    - [x] ~~Project Timeline (Abhinav Binu)~~
    - [x] ~~Functional & Nonfunctional Requirements (Jaden Kuriakose)~~
    - [x] ~~Diagrams (Abhinav Binu)~~

- [x] ~~Architecture Design (Dev Sheth)~~
- [x] ~~Comparison w/ systems (Jaden Kuriakose)~~
- [x] ~~Test Cases (Ahmed Fouad)~~
- [x] ~~Figma (Raisa Reza)~~
- [x] ~~Challenges, Lessons, & Conclusions (Raisa Reza)~~

- [ ] Presentation recording (max 15 minutes)
- [ ] Final submission to GitHub
- [ ] Final submission to eLearning

Project Report: Financial QA Chatbot

By: Jaden Kuriakose, Dev Sheth, Dev Singh, Raisa Reza, Abhinav Binu, & Ahmed Fouad

Github Repository: https://github.com/jadenkuriakose/3354-team6

Task Delegation

Jaden Kuriakose

I contributed to the slideshow and report documentation for the project. On top of that, I have helped with the assumptions, feedback, and requirements sections of the report from deliverable I and did the scope document. I additionally helped with writing unit tests and worked on data cleaning and processing tasks and LLM handling functions.

Ahmed Fouad

I contributed to the slideshow the slides about “Testing and Validation” along with presenting them and to the report in part to the software & hardware costs and project timeline by contributing to what other members have written before. I have helped with the choice of what process model is suitable for this project.

Abhinav Binu

I contributed on the hardware and personnel cost estimates, where I looked at what it would realistically cost to run something like this in a real world setting. I also improved the sequence diagrams so they better showed how everything in the system connects and works together. For presenting, I covered the project timeline and explained the use case and sequence diagrams. I also helped review parts of the report and made small edits wherever needed to keep everything consistent.

Raisa Reza

I contributed the presentation slides which portray the UX design, made by using Figma, for our frontend design idea into a diagram explaining what each feature does and how it coordinates with our overall project idea/use cases. Additionally, I improved the use-case diagram to have

goals/actions that the user can complete using the software by making two diagrams for both the login page and the chatbot interface, and helped write the project scheduling/timeline for the deliverable report and conclusions/lessons learned for both the deliverable and presentation slides.

Assumptions

1. We assume that we will be able to use LLMs accessed by API from a provider for free model access or potentially access locally hosted models through a library like oLlama
2. We assume we can vectorize and process saved values into embeddings in a free vector database using an embeddings model that will convert chunks into embeddings for semantic search
3. The system is expected to only process a certain amount of requests per time frame as we likely will not have high usage access to tools or models.
4. We assume that our data will be saved in such a way that it is connected to a user and can only be accessed by that specific user.
5. The system is assumed to only be used when internet access is provided, that way LLM APIs and services can be accessed.
6. The system assumes data will only be passed using queries, csv files, pdfs, or images.

Proposal Feedback

The feedback for the proposal provided for deliverable 1 was that we needed to define use cases and problems the project is solving. The initial proposal focused more on what the chatbot does and how it responds to prompts architecturally. As such, the provided implementation and motivation was incorrect and we were told to refine these sections for this deliverable.

For the project motivation, we chose this project because we are interested in finance and business type documents as a group and wanted to work on a chatbot or agent that can analyze these documents. This project would help us learn how to integrate AI frameworks and backend functionality to process data for AI responses. We believe that this project can be used by us and others to analyze financial documents without having to spend too long looking over numbers or reading complex sections.

For the project implementation, the use case is centered around allowing users to analyze financial documents without deep background knowledge or spending too much time. This is a useful case because financial documents often have lots of data, numbers, pages, or difficult jargon that make it difficult to understand for people. By using AI, we can process these long documents and numbers and allow users to use AI to understand and analyze this information

using the proper background. As a result, the product will speed up the process of reviewing financial documents and will act similar to an analyst or consultant reviewing your information and providing quick answers and guidance.

Process Model

Our team has selected a "Scrum" process model to execute this project. We view our time as highly flexible and adaptable and we also view our project as one that would need reiterations and feedback since AI models seem to go through similar iterations and improvements as time goes by. We would need feedback from testing and eventually from instructors to our next sprint goals. We also think the idea of having "sprints" resonates with how we are going to focus our attention into solving specific problems before moving on to tackle others avoiding distractions and ensuring progress one bit at a time. We also resonated with the idea of rapid feedback that would happen in daily scrum meetings ensuring everyone is still on track especially with our differing experience levels eliminating any obstacles or hurdles in the way, as a team, through discussions, reassurances, advice and resources.

Functional Requirements

1. The system shall process data in pdf, csv, text, and image format
2. The system shall pass context onto the LLM through RAG pipeline to generate responses
3. The system shall send user queries and context to the LLM
4. The system shall validate responses to make sure that output is based on provided context
5. The system shall store session memory and data securely
6. The system shall authorize users using login
7. The system shall display responses in the UI

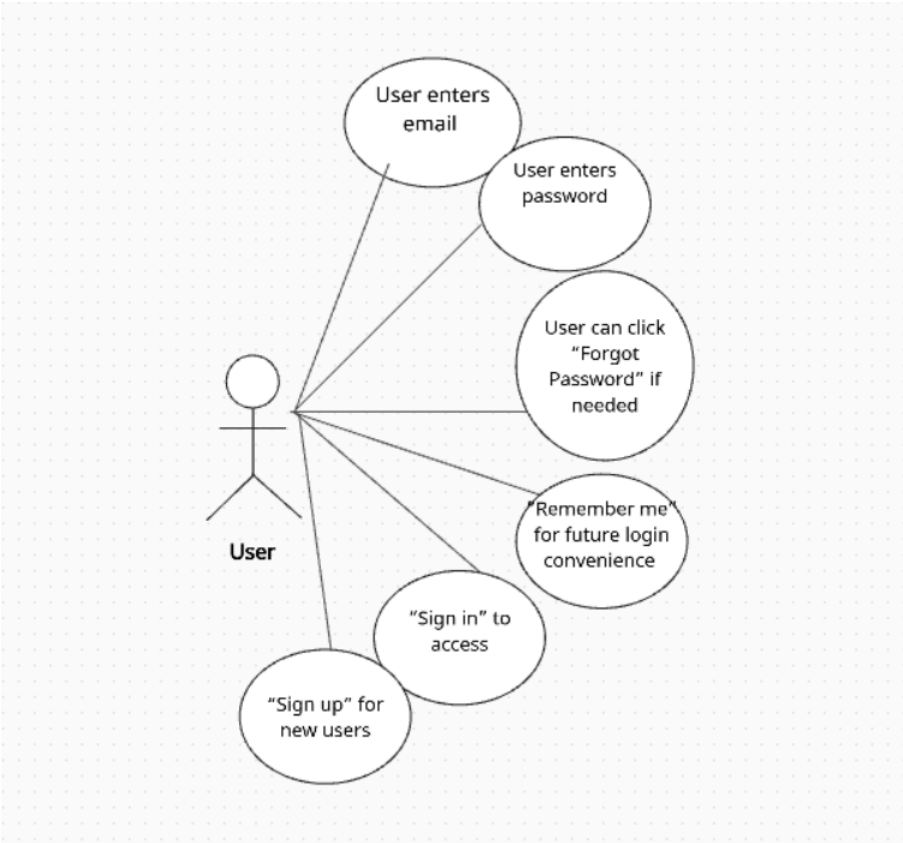
Non-Functional Requirements

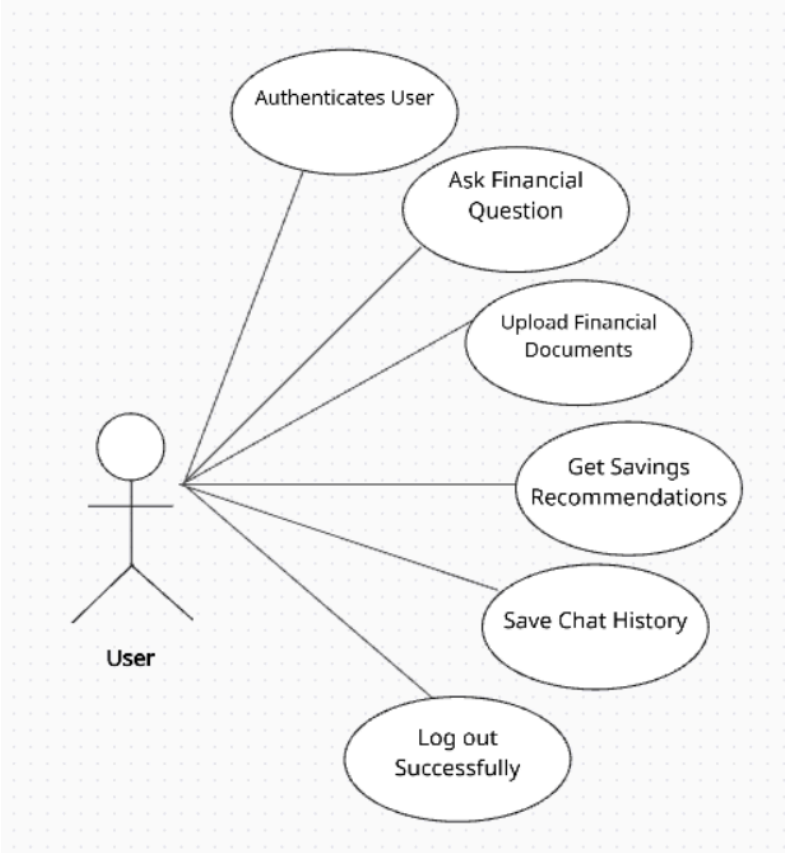
1. Usability Requirement: The system shall allow users to upload documents with drag and drop or selecting files from their device and easily interact with the chatbot
2. Performance Requirement: The response shall be returned in under 5 seconds without excess documents and under 1 minute with excess documents
3. Space Requirement: The system shall be able to handle documents of up to 25 MB minimum
4. Dependability Requirements: The system shall have consistent and reliable access to LLM API tokens in order to maintain high uptime to ensure consistent chatbot responses
5. Security Requirements: The system shall allow users to login using their credentials and maintain session and user security to protect sensitive financial data
6. Environmental Requirement: The system will operate in an environment with internet access so that it can communicate with APIs and cloud services used for the chatbot and document processing
7. Operational Requirement: The system shall support communication between the frontend interface and backend services through API requests to ensure the chatbot functions properly
8. Development Requirement: The system shall be developed using commonly supported

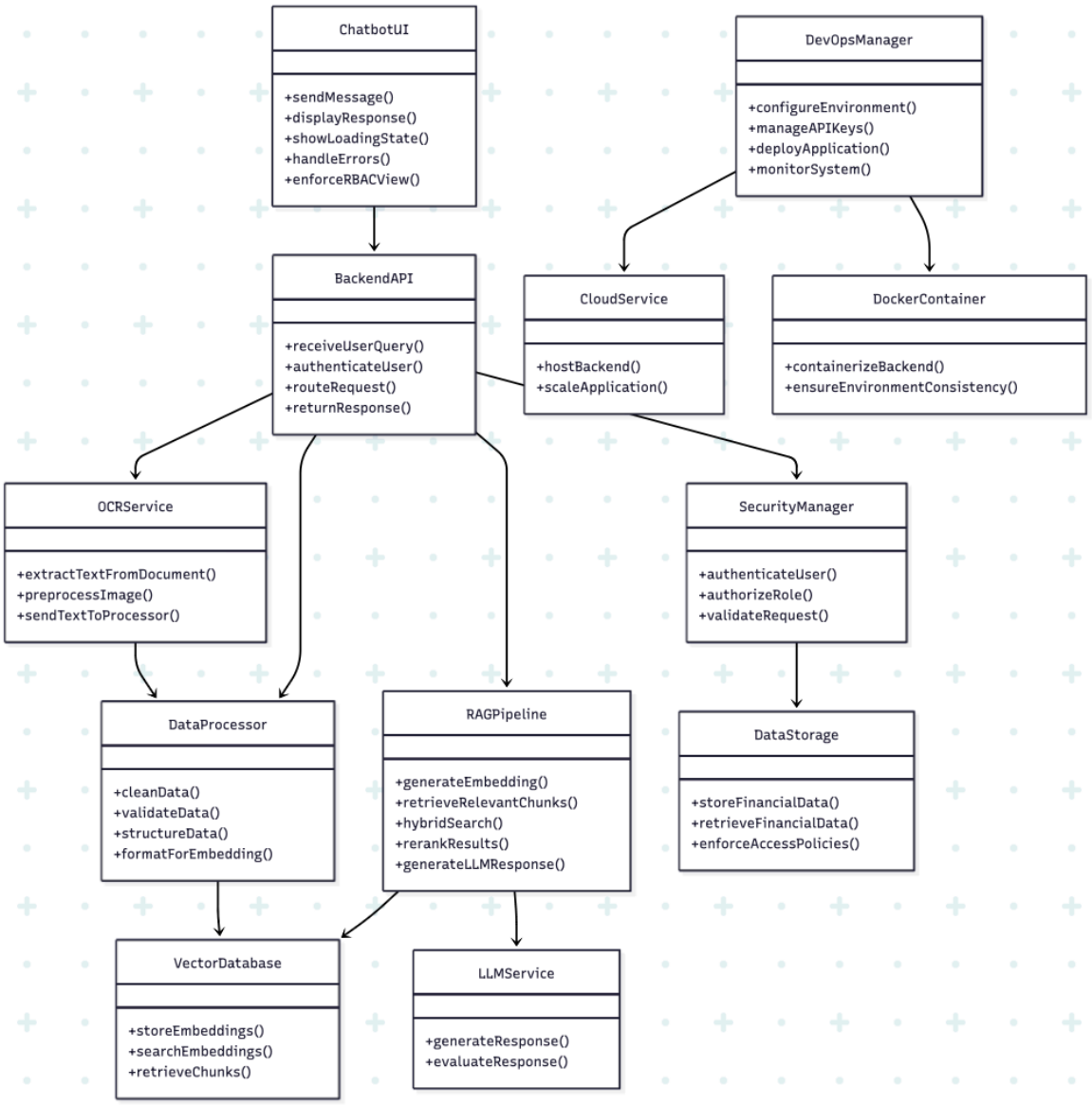
programming languages and tools so that the system can be easily maintained and updated by the developers
9. Regulatory Requirement: The system shall follow proper and applicable financial data regulations and privacy standards when processing and storing user uploaded documents
10. Ethical Requirement: The system shall generate responses based only on the provided financial documents and avoid producing misleading or fabricated financial information
11. Accounting Requirement: The system shall maintain logs of system interactions and document processing activities in order to support auditing and monitoring of the system
12. Safety/Security Requirement: The system shall protect sensitive financial data using secure authentication and access controls in order to comply with applicable privacy and data protection regulations

Diagrams

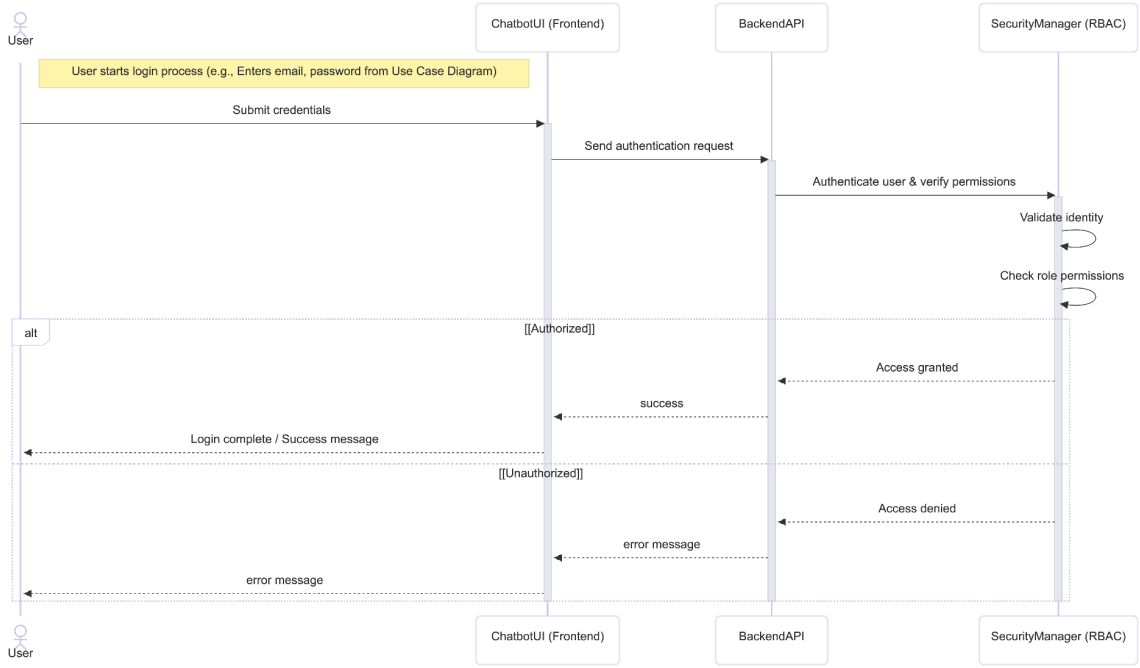
Use-Case Diagrams (For Login Functionality and Chatbot)


User enters email
User enters password
User can click "Forgot Password" if needed
"Remember me" for future login convenience
"Sign in" to access
"Sign up" for new users
User

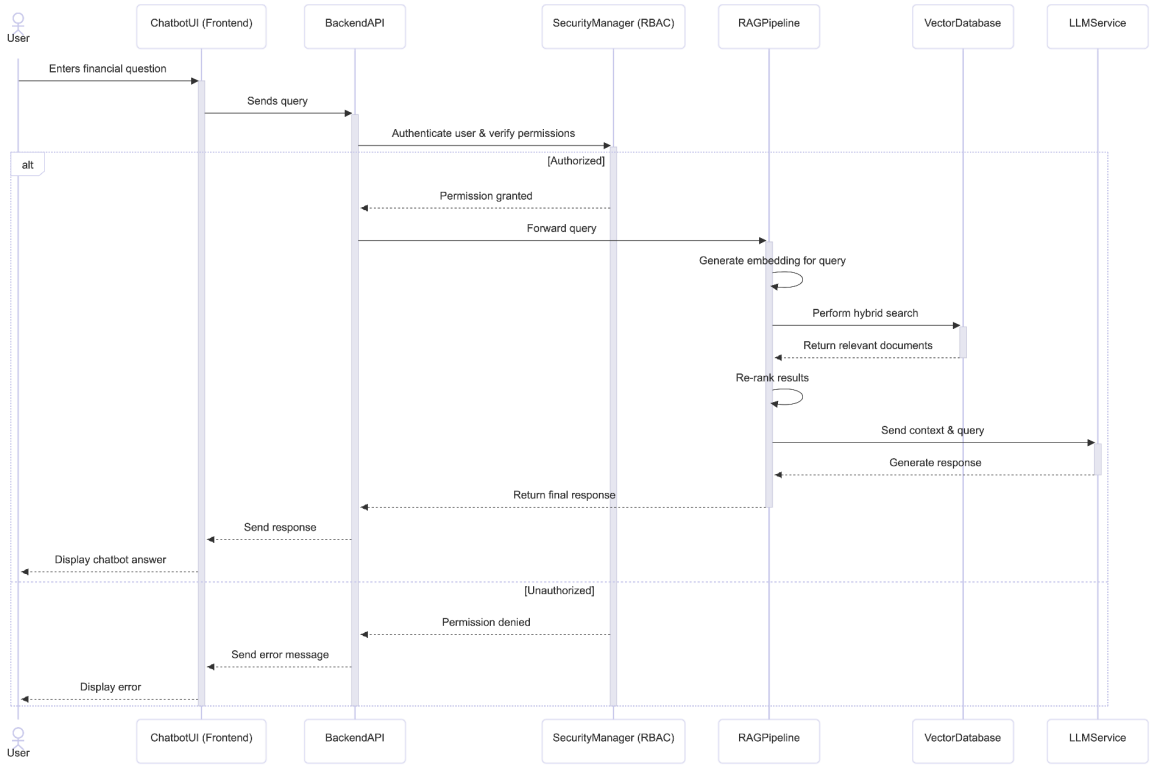

Authenticates User
Ask Financial Question
Upload Financial Documents
Get Savings Recommendations
Save Chat History
Log out Successfully
User


ChatbotUI
+sendMessage()
+displayResponse()
+showLoadingState()
+handleErrors()
+enforceRBACView()
DevOpsManager
+configureEnvironment()
+manageAPIKeys()
+deployApplication()
+monitorSystem()
BackendAPI
+receiveUserQuery()
+authenticateUser()
+routeRequest()
+returnResponse()
CloudService
+hostBackend()
+scaleApplication()
DockerContainer
+containerizeBackend()
+ensureEnvironmentConsistency()
OCRService
+extractTextFromDocument()
+preprocessImage()
+sendTextToProcessor()
SecurityManager
+authenticateUser()
+authorizeRole()
+validateRequest()
DataProcessor
+cleanData()
+validateData()
+structureData()
+formatForEmbedding()
RAGPipeline
+generateEmbedding()
+retrieveRelevantChunks()
+hybridSearch()
+rerankResults()
+generateLLMResponse()
DataStorage
+storeFinancialData()
+retrieveFinancialData()
+enforceAccessPolicies()
VectorDatabase
+storeEmbeddings()
+searchEmbeddings()
+retrieveChunks()
LLMService
+generateResponse()
+evaluateResponse()

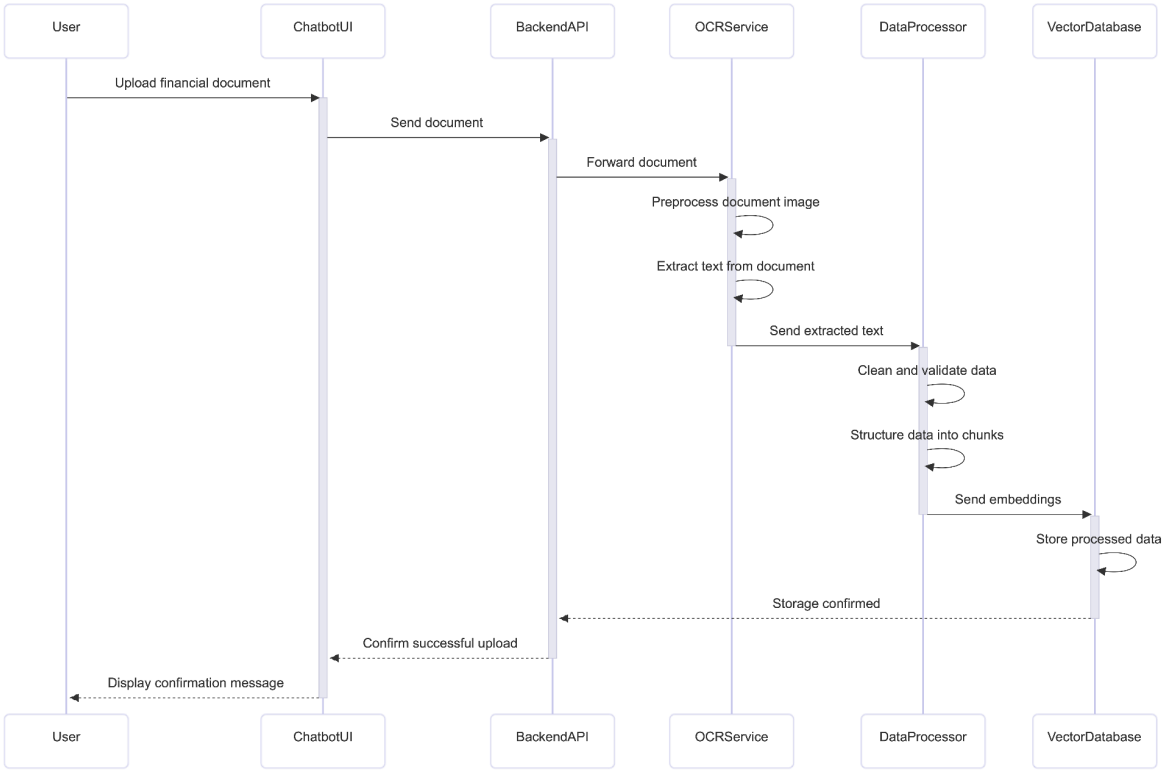
A) User Authentication and Login



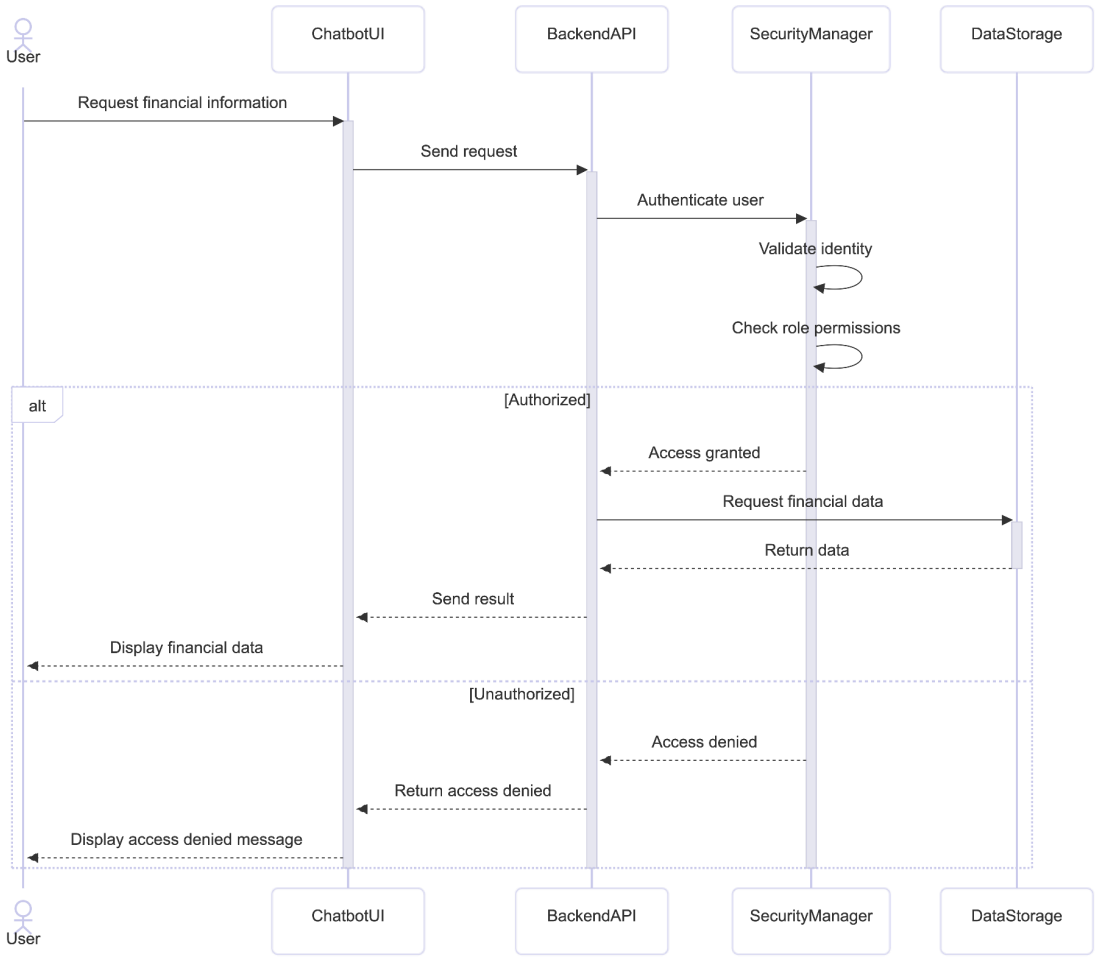
B) User Asks for Financial Question



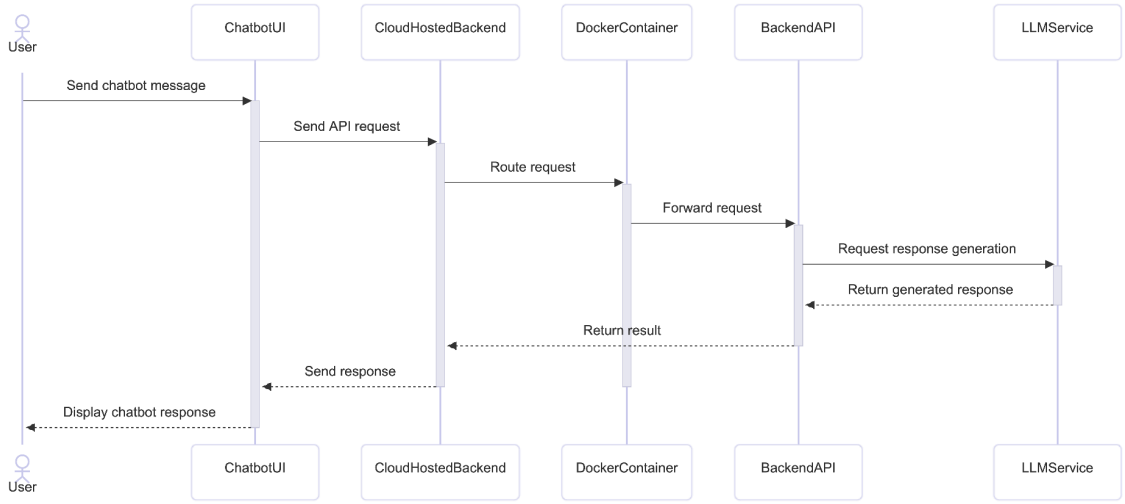
B) User uploads a financial document for processing



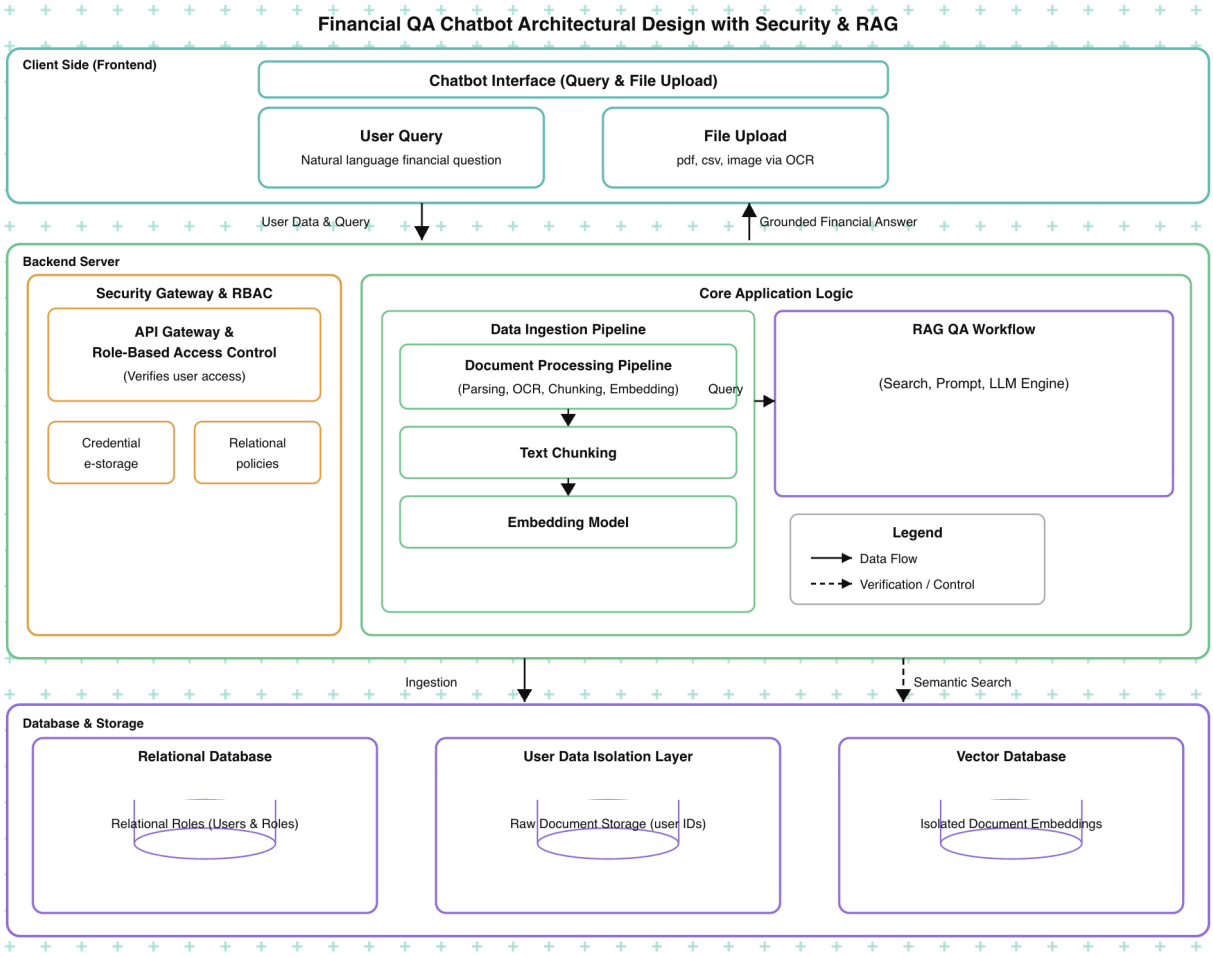
C) Secure Data with RBAC



D) Chatbot Deployment and API Communication



Architectural Design Diagram



Cost & effort estimation method

3.2 Cost, Effort, and Pricing Estimation

Method:

Because our product is very dynamic and data-driven, we are utilizing Function Point Analysis (FPA). Our system's functionality—more especially, its ability to accept user documents, query an LLM, and provide precise outputs—is what makes it so valuable. Regardless of the particular programming languages or frameworks we employ, this approach enables us to quantify the system size, development effort, and cost.

To apply the function point estimation, the following steps will be used:

1. Determine function category count for: User input, user output, user queries, data files and relational tables, and external interfaces.
2. Determine complexity using the weighting table.
3. Compute Gross Function Point (GFP).
4. Determine Processing Complexity (PC) based on the 14 system characteristics.
5. Compute Processing Complexity Adjustment (PCA).
6. Compute Function Point (FP) using the formula: FP = GFP * PCA.
7. Calculate the estimated effort using the formula: E = FP / productivity.
8. Calculate the project duration using the formula: D = E / team size.

Application of Function Point:

Step 1: Determine function category count

First, we need to categorize the specific functions of the Financial QA Chatbot:

- **User Input** includes inputs that update internal data or trigger processing, such as: creating an account, logging in, uploading a document (PDF, CSV, Image), submitting a text query to the chatbot, deleting a document, and clearing session memory. Total count: 6.
- **User Output** includes outputs produced by the system, such as: displaying the AI chatbot response, displaying validation/error messages, outputting the extracted text status from a PDF, and exporting a chat log. Total count: 4.
- **User Queries** include operations that retrieve data without updating it, such as: viewing past chat session history, fetching the list of uploaded documents, and checking user authorization levels. Total count: 3.
- **Data Files and Relational Tables** are the internal databases of the system, including: the Users/RBAC table, the Chat Session/Memory table, the Document Metadata table, and the internal Vector Embeddings database. Total count: 4.
- **External Interfaces** are external APIs and services our system communicates with but does not maintain, such as: the LLM API (Gemini) and the external authentication provider (Auth0/Custom). Total count: 2.

Steps 2-3: Determine complexity and calculate GFP

We will assume an "Average" complexity for all functions, as they involve standard API integrations and CRUD operations typical of an AI web application.

Function Category	**Count**	**Complexity (Average)**	**Count × Complexity**
1. # of User Inputs	6	4	24
2. # of User Outputs	4	5	20
3. # of User Queries	3	4	12
4. # of Data Files and Relational Tables	4	10	40
5. # of External Interfaces	2	7	14
Gross Function Point (GFP)			**110**

Step 4: Determine processing complexity (PC)

We evaluated the 14 standard processing complexity questions on a scale of 0 (no influence) to 5 (essential). Because this system requires heavy data communications with APIs, solid performance for LLM response times, and secure handling of financial data, the total sum of these factors is 35.

Step 5: Compute processing complexity adjustment (PCA)

We calculate PCA using the standard formula:

PCA = 0.65 + 0.01 * (PC)

PCA = 0.65 + 0.01 * (35) = 1.00

Step 6: Compute function point (FP)

Using the formula:

FP = GFP * PCA

FP = 110 * 1.00 = 110 FP.

Step 7: Calculate the estimated effort

We will assume that the productivity of the development team is 10 function points per person-week.

Using the formula:

E = FP / productivity

E = 110 / 10 = 11 person-weeks.

Step 8: Calculate the project duration

The project development team consists of 6 people. This calculation focuses strictly on development effort, excluding planning and documentation phases.

Using the formula:

D = E / team size

D = 11 / 6 = 1.83 weeks (approximately 2 weeks of dedicated coding).

Project Scheduling / Timeline

Start Date: February 8, 2026, End Date: May 3, 2026 (estimated)

Justification: The project is estimated to take about 8 weeks. This allows enough time to go through each phase of preparations & specifics, project proposal, deliverables, and practicing presentations properly without rushing, while still keeping the work manageable alongside other coursework. Each deliverable is given roughly one to two weeks, with the final phase including extra time for testing and adjustments.

1. Weekends will be counted into our schedule, our work can happen on both weekdays and weekends depending on each team members' availability and as a safety net if someone is falling behind or needs extra time.
2. Work is done flexibly, if we count our consistent communication. Some days may have little to no work, while others may involve longer work sessions depending on deadlines and team members' tasks which can range from two to four hours per week.

Project Scheduling / Timeline (revisited)

Start Date: February 8, 2026, End Date: May 3, 2026

The project is estimated to take approximately 8 weeks in a real-world development environment. Those 8 weeks include time required for the team (or company) to design, develop, test, and prepare the Financial QA Bot for deployment as a complete product.

This timeline visits multiple phases in the software development lifecycle such as requirements analysis, system design, implementation, integration of external APIs (such as LLM services), testing, debugging, and final deployment preparation. Additional time is added for validating the RAG pipeline and ensuring data security.

We estimated the development effort in the Function Point Analysis to approximately 2 weeks of dedicated coding but the full life cycle would extend to allow time for all non-coding activities.

We assume a standard work schedule of 8 hours per day, 5 days per week and weekends are not included as to match industry practice.

Now going into a detailed timeline, let us look closer at the phases.

Requirements Analysis (1 week): Defining system functionality, identifying supported file types (PDF, CSV, images), and outlining user interactions with the chatbot. Additional time is needed to clarify data handling and security requirements for financial documents and collect information about non functional requirements such as time needed for response, data privacy (ensuring this data is not going to train the LLM models behind our Q/A bot), citation & traceability (to avoid legal liability).

• **System Design (1 to 1.5 weeks):** Designing the system architecture, including the RAG pipeline, API integrations, database structure, and user authentication mechanisms. This phase also includes designing the UI workflow and interaction flow between components.

• **Implementation (2 weeks):** Developing core functionalities such as document upload, PDF parsing, embedding generation, LLM query handling, and frontend-backend communication. This aligns with the estimated coding effort derived from Function Point Analysis.

• **Integration (1 to 1.5 weeks):** Integrating external services such as LLM APIs and vector databases, and ensuring smooth communication between system components. This phase may require additional time due to API limitations, response validation, and debugging integration issues.

• **Testing and Debugging (1.5 to 2 weeks):** Conducting unit testing, validating system outputs, handling edge cases (e.g., invalid files or incorrect queries), and ensuring system reliability and performance.

• **Deployment Preparation (few days):** Finalizing the system for deployment, including performance tuning, security checks, and preparing documentation or user guidelines.

Unit Tests

For this deliverable, we will test our pdf parsing function. The way that we will test this is by passing proper paths and improper paths to test pdf documents. From there, we can test the functions ability to handle these paths whether valid or invalid and see if the parsed text matches the text we read in the pdf as expected. We can parse this text using pdfplumber and then we can use pytest for the testing. We can test with various financial documents or simple statements that we have access to and validate them to ensure this function is working properly.

PDF parsing function code

```
# Function Code
from PyPDF2 import PdfReader

def extractTextFromPdf(filePath):
    if not filePath:
        raise ValueError("file path cannot be empty")

    reader = PdfReader(filePath)
    extractedText = []

    for page in reader.pages:
        pageText = page.extract_text()
        if pageText:
            extractedText.append(pageText)

    return "\n".join(extractedText).strip()
```

Unit Test Cases – PDF Parsing Function

The extractTextFromPdf() function was tested using the PyTest testing framework.
The goal of these tests was to ensure the function correctly extracts text from valid PDF files and properly handles invalid inputs.

Test Case	INPUT	EXPECTED RESULT	ACTUAL RESULT
Valid PDF Path	sample.pdf	Text successfully extracted from the PDF	PASSED
Invalid File Path	fake_file.pdf	Function raises FileNotFoundError	PASSED
Empty File Path	“”	Function raises ValueError	PASSED

Testing Tool: PyTest
Library Used: PyPDF2
All unit tests were executed successfully, confirming that the PDF extraction function handles both valid and invalid inputs correctly.

```
PS C:\Users\Dev\Documents\GitHub\3354-team6> python -m pytest test_pdf_parser.py -v
========================================= test session starts =========================================
platform win32 -- Python 3.14.2, pytest-9.0.2, pluggy-1.6.0 -- C:\Python314\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\Dev\Documents\GitHub\3354-team6
collected 3 items

test_pdf_parser.py::test_valid_pdf PASSED                                                    [ 33%]
test_pdf_parser.py::test_invalid_path PASSED                                                 [ 66%]
test_pdf_parser.py::test_empty_path PASSED                                                   [100%]
```

Hardware Costs

processing is handled through APIs and cloud-based services. During development, all team members will be using their personal laptops or desktops, so there is no additional cost for dedicated hardware at this stage. These devices are sufficient for running the frontend, backend, and testing environments locally. If the system were to be deployed beyond development, we would likely rely on cloud infrastructure rather than physical hardware. This would include services such as virtual machines or container hosting platforms, which eliminate the need to purchase and maintain physical servers. As a result, there are no upfront hardware costs for deployment. Overall, the estimated hardware cost for this project is $0, since we are using existing personal devices and do not require any specialized equipment like GPUs or on-premise servers.

Hardware Costs (revisited)

In a real-world deployment environment, the Financial Q/A Bot would rely on cloud-based infrastructure to support scalability, availability, and performance. The system would require virtual machines or container-based services to handle backend processing, API communication, and data storage. We also assume the software development company would already have computers and a setup environment for writing code so no need to include things like monitors, keyboards, mouses etc.

The primary hardware cost therefore would be the cloud compute instances, storage, and networking resources. We might just use a cloud provider like AWS or Microsoft Azure. These small to medium virtual machines would cost 30-80 a month depending on performance requirements. An additional storage for uploaded documents and vector embeddings would be 5-20 a month depending on our scale and data volume.

Highlighting that we mainly rely on external APIs for LLM processing, no specialized GPU is needed.

This brings us to a total of 35-100 a month if this was to be developed by a real software company.

Software Costs

This system will use free and open source software for extraction, testing, and vector databases. Specifically, we can use libraries and tools like Qdrant, FAISS, and pytest for free for these functions. On top of that, we can use custom authentication methods or use managed tools like Auth0 for user verification. We will end up spending money on cloud infrastructure and LLM costs in production, which would likely be about $20-100/month, depending on token usage, user counts, and student pricing. For now, we will host locally and use APIs for Gemini under the free tier and as such will be using these services for free. In production under paid tiers, gemini-3.1-flash-lite-preview would be $.25/1M tokens input and $1.50/1M tokens of output [1].

Software Costs (revisited)

In a real world deployment, the Financial Q/A bot would be relying on some free tools (open source) along with some paid cloud services. A real development team can use Qdrant or FAISS for our database vector search. The real costs would arise from hosting, managed services, and API usage. LLM APIs would cost the team 0.25$ per 1 million input tokens and 1.5$ per 1 million output tokens. Authentication and database hosting would typically cost from 10$ to 50$ depending on scale and usage.

To get a closer estimation of scale and prices, 25 users per day using 5 to 8 queries would be 8 to 15 million tokens per month making the API cost about 10 to 25 a month. A heavier usage would be 25 users asking 16-20 queries (premium users, a corporate deal for example) would result in 25 to 40 million tokens which in turn would cost the software development company about 50$ to 70$ a month on these tokens.

Personnel Costs Estimation

Since this project is being developed as part of a course, there are no direct personnel costs involved. All team members are contributing their time without financial compensation. However, to estimate what the cost would look like in a real-world scenario, we can assign approximate hourly rates based on entry-level software development roles. Assuming an average rate of $25–$35 per hour per team member and estimating that each member contributes around 40–60 hours over the course of the project, the total effort can be calculated. With six team members, this results in:

- Average hours per person: 50 hours
- Average rate: $30 per hour
- Total estimated cost: 6 members × 50 hours × $30 per hour = $9,000

This is a rough estimate and would vary depending on experience level, project complexity, and actual time spent. In a professional setting, additional roles such as project managers or QA engineers could increase this cost further. Even though there is no actual financial cost for this project, this estimate gives a realistic idea of what similar work might cost in an industry environment.

Pricing Model

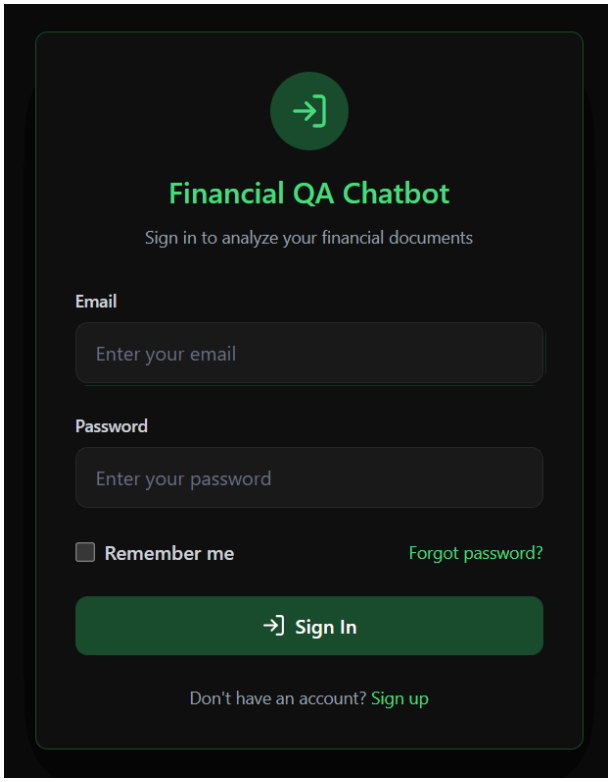
This Financial Q/A Bot can be priced with a subscription based model where users pay monthly according to how heavy is their usage of the system, particularly the number of documents processed and queries made. We will have a popular subscription for everyday users and another for professionals like accountants or lawyers that would be using the system to a greater capacity. Then we will also introduce a third for enterprises and corporates. The basic tier can be 7$ a month, premium is 20$ and then corporate tier is 50$. This pricing structure accounts for operational costs such as cloud hosting, storage, and LLM API usage, while also ensuring scalability and profitability as the number of users increases. This pricing model allows the product to gain back the development costs in about 30 months assuming 100 basic tier users. The strategy is that premium and basic users are to balance the costs and spendings while the corporate tier is one that makes the profit.

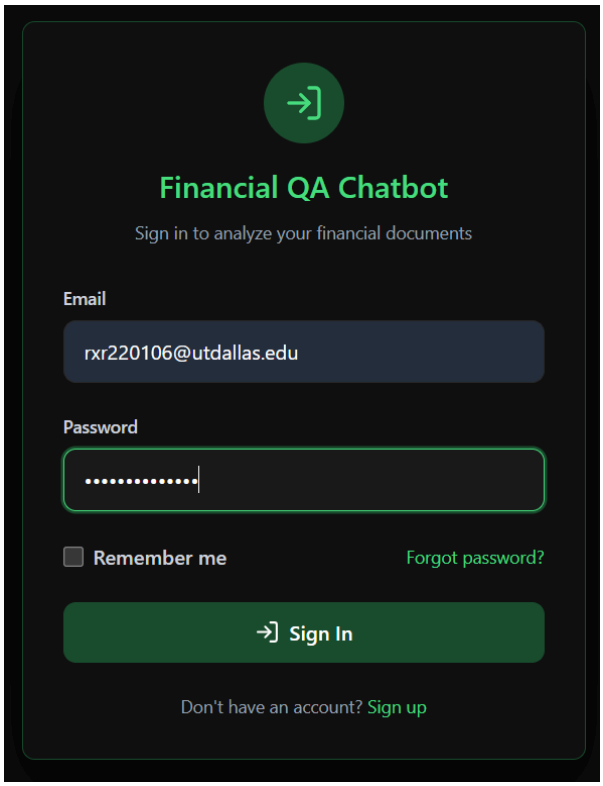
Comparison with Other Systems

Other systems like ChatGPT exist that use LLMs to process data from various formats for response. These systems include tools that can be used to search for context or data from the internet. While our system might not be able to do that, our system is designed to provide context specifically on financial data using RAG to ensure responses are supported by context. On top of that, our system uses an LLM API and secures data meaning that PIIs (personally identifiable information) and context in financial documents won't be used to train an LLM, as this is often a privacy issue [2].

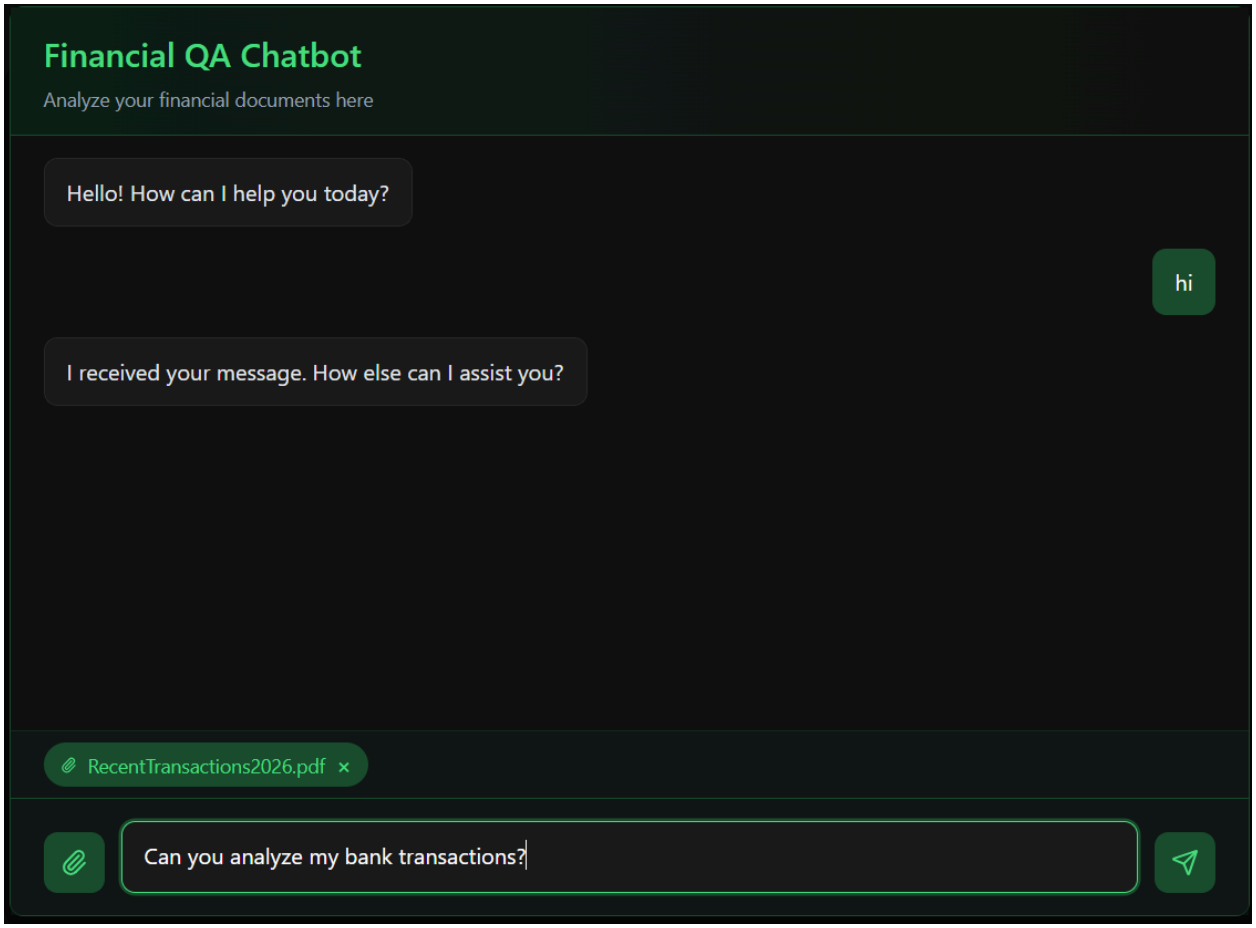
Another alternative would be a financial tool like excel or sheets. These systems can be used for data visualization and financial calculations. These tools help keep track of data but often require data to be filled out by users. These systems additionally require more effort and knowledge to use. Our system on the other hand is able to read data and transform it into easy to read and understand insights quickly with less effort and background financial knowledge.

Prototype of application (Figma)



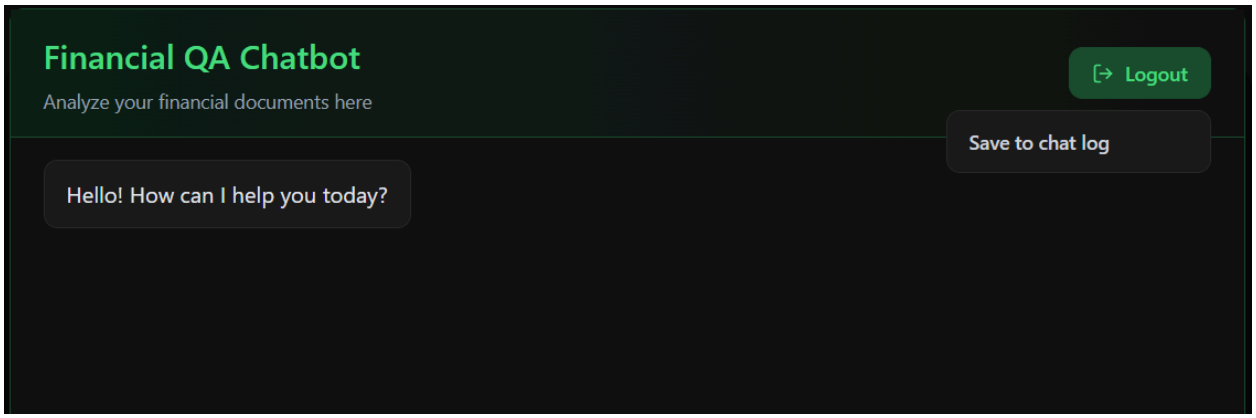


The functionality of the UX design includes a login screen that users see before accessing the chatbot, essentially telling users to sign in to analyze their financial documents or speak to the chatbot. Users can enter their email and password into the provided input spaces, with an option to check "Remember me" for convenience or click "Forgot password?" if needed as visualized in most login pages when performing authorization. Once user credentials are given, users can now press the sign in button to access the bot. For new users, a "Sign up" link is available at the bottom of the screen. This screen reflects one of the functional requirements of our Chatbot, which makes sure that only authorized users can access and interact with their uploaded financial documents.



Here is a demonstration and UX design using Figma showing the components of our financial analysis chatbot. At the bottom of the screen, users can type their questions into a text input space. There is also a provided file attachment button that allows users to upload financial documents such as PDFs, csv's, or images of minimum 25 MB. Once a file is attached, it appears as a tag above the input space (as shown with "RecentTransactions2026.pdf" with the paperclip icon). The user can then submit a query/question related to the uploaded document, such as asking the chatbot to analyze their bank transactions in this scenario. This design reflects the core functionality of the system, which allows users to upload documents and interact with the chatbot directly. Additionally, there is a logout button on the top-right corner so the user can save their existing conversation, and exit the conversation whenever they please by logging out.

(next page)



Conclusion / Project Evaluation

I. Some changes that we needed to make were in how we defined our motivation, implementation, and use cases. The feedback from Deliverable I pointed out that our original proposal focused too heavily on the technical architecture, or rather the "how" of the chatbot rather than "what" were the actual problems it was solving. We rewrote our motivation and implementation explanations to better explain why this project is useful, which was essentially helping people analyze financial documents without requiring deep financial knowledge or spending excessive time reading through complex data.

II. The justification for these changes is that our project proposal should clearly communicate why the project is being built and what problems it aims to solve, not just how it works technically. We rewrote the motivation to explain that we chose this project because we are interested in financial documents and wanted to build a tool that helps users analyze them without needing deep financial background knowledge and help save time reading through complex data. We also rewrote the implementation section to focus on the actual use case, which is again allowing users to analyze financial documents without needing background knowledge or spending too much time. Financial documents often contain large amounts of data, complex jargon, and lengthy sections that are hard to get through, so by using AI to process and interpret that information, the system can act similarly to an analyst or consultant that quickly reviews your documents and provides clear answers when receiving the results.

Works Cited

[1]

“Gemini Developer API pricing,” *Google AI for Developers*, 2025. https://ai.google.dev/gemini-api/docs/pricing#gemini-3.1-flash-lite-preview (accessed Mar. 29, 2026).

[2]
OpenAI, “Does OPEN AI API use API data for training?,” *OpenAI Developer Community*, Feb. 28, 2024. https://community.openai.com/t/does-open-ai-api-use-api-data-for-training/659053 (accessed Mar. 30, 2026).

[3] “Pricing - linux virtual machines: Microsoft Azure,” Pricing - Linux Virtual Machines | Microsoft Azure, https://azure.microsoft.com/en-us/pricing/details/virtual-machines/linux/ (accessed Apr. 1, 2026).

[4] Amazon EC2 – secure and resizable compute capacity – aws, https://aws.amazon.com/ec2/pricing/ (accessed Apr. 2, 2026).